Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

A SKILL BASED EVALUATION REPORT

SUBMITTED BY

JEBARSON CALEB D (URK24CS1038)

TALARI S VAMSHI KRISHNA (URK24CS1025)

JASPER FREDDY PAUL R S (URK24CS1118)

PAVIN MUTHU G (URK24CS1017)

COURSE CODE

23CS2048

COURSE NAME

WEB TECHNOLOGY

OCTOBER 2026

DIVISION OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

HIRESENSE: AI-ASSISTED RESUME ANALYSIS AND JOB MATCHING PORTAL

A REAL TIME APPLICATION REPORT

Submitted by

JEBARSON CALEB D (URK24CS1038)
TALARI S VAMSHI KRISHNA (URK24CS1025)
JASPER FREDDY PAUL R S (URK24CS1118)
PAVIN MUTHU G (URK24CS1017)



DIVISION OF COMPUTER SCIENCE AND ENGINEERING

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be-under Sec-3 of the UGC Act, 1956) Karunya Nagar, Coimbatore - 641 114. INDIA

OCTOBER 2026

ABSTRACT

Recruitment today is dominated by two manual, slow activities: job seekers guess whether their resume will pass an Applicant Tracking System (ATS), and recruiters read hundreds of resumes to find a handful of suitable candidates. **HireSense** is a full-stack web application that automates both sides of this loop with an AI-assisted resume analysis and job matching engine.

The application supports three roles. A **job seeker** uploads a resume (PDF, DOCX or TXT); the server extracts its text and asks a large language model (Groq-hosted *openai/gpt-oss-120b*, or a local *llama3.2* model through Ollama) for a structured report containing an ATS score, detected skills, years of experience, strengths, weaknesses, concrete improvement suggestions and suitable job roles. Every open job is then ranked for the seeker using a fast hybrid score (required-skill overlap plus TF-IDF cosine similarity), and an on-demand AI fit check explains the decisive matches and gaps. A **recruiter** posts jobs, sees applicants ranked by match score, discovers suitable candidates who have not applied, updates application status and downloads original resumes. An **administrator** monitors platform statistics through charts and manages users and job posts. If the AI provider is unreachable or returns malformed output, a rule-based engine (a 220-skill dictionary with aliases and heuristic ATS checks) takes over, so the portal always responds.

The front end is a React 18 single-page application built with Vite, React Router, Axios and Chart.js, with a responsive light/dark theme and client-side form validation. The back end is a Node.js and Express 5 REST API secured with JSON Web Tokens, bcrypt password hashing, role-based route guards, Helmet and CORS, with server-side validation through express-validator and Multer for size- and type-restricted uploads. Data is persisted in **MongoDB** through Mongoose schemas for four collections (users, resumes, jobs and applications), including the original resume file, the embedded AI analysis and a per-application score breakdown. The system follows a three-tier client–server architecture and is deployed on Vercel (static front end plus a serverless API) with MongoDB Atlas.

Functional testing of 23 API scenarios (validation, authentication, authorisation, upload limits and AI analysis) produced the expected result in every case, and the interface was verified on desktop and mobile viewports in both themes.

Keywords: MERN stack, React, Node.js, Express, MongoDB, REST API, JWT, large language model, ATS score, resume parsing, TF-IDF, job matching.

TABLE OF CONTENTS

CONTENTS **PAGE NO.**

CHAPTER 1: INTRODUCTION 1
1.1 Background of the Project 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
CHAPTER 2: LITERATURE SURVEY 3
2.1 Review of Existing Websites / Web Applications 3
2.2 Summary of Existing Approaches 4
2.3 Research Gap or Limitations of Existing Systems 4
CHAPTER 3: METHODOLOGY AND SYSTEM DESIGN 5
3.1 Proposed Methodology 5
3.2 System Architecture 6
3.3 Website Structure and Navigation Flow 6
3.4 UML Diagram – Use Case Diagram 7
3.5 Database Design 8
3.6 Tools and Technologies Used 9
3.7 Module Description 10
3.7.1 Authentication and User Management Module 10
3.7.2 Resume Upload and AI Analysis Module 10
3.7.3 Job Management Module 10
3.7.4 Matching and Application Module 11
3.7.5 Administration Module 11
3.7.6 AI Provider and Fallback Module 11
CHAPTER 4: IMPLEMENTATION AND RESULTS 12
4.1 Implementation Details 12
4.1.1 Front-End Development 12
4.1.2 Back-End Development 13
4.1.3 Database Connectivity 14
4.2 Screenshots of the Web Application 16
4.3 Sample Inputs and Outputs 24
4.4 Testing and Results 26
4.4.1 Unit Testing 26
4.4.2 Cross-Browser and Responsiveness Testing 26
4.4.3 Form Validation and Functional Testing 27

CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENT 28
5.1 Summary of the Work 28
5.2 Conclusion 28
5.3 Future Enhancements 28
REFERENCES 30
SAMPLE OUTPUT SCREENSHOT 32
ANNEXURE: HACKNEX 2026 PARTICIPATION CERTIFICATES 38

LIST OF FIGURES

Figure 3.1: Resume analysis and job matching pipeline....5
Figure 3.2: Three-tier system architecture of HireSense....6
Figure 3.3: Sitemap and navigation flow....7
Figure 3.4: Use case diagram....8
Figure 3.5: Entity–relationship diagram of the MongoDB collections....9
Figure 4.1: users collection – passwords stored only as bcrypt hashes....14
Figure 4.2: jobs collection – a job post created by a recruiter....15
Figure 4.3: applications collection – match score and embedded score breakdown....15
Figure 4.4: Landing page....16
Figure 4.5: Login page....17
Figure 4.6: Job seeker dashboard with top job matches and resume snapshot....18
Figure 4.7: AI resume analysis report – ATS score, skills, issues and suggested fixes....19
Figure 4.8: Job matches ranked by match score....20
Figure 4.9: Recruiter dashboard – job posts with applicant statistics....21
Figure 4.10: Applicants for a job ranked by AI match score....22
Figure 4.11: Admin platform overview with charts....23
Figure 4.12: Responsive layout on a mobile viewport (390 px wide)....24
Figure 4.13: Post a job form showing a client-side validation message....25
Figure 4.14: Registration form showing a server-side validation error (duplicate e-mail)....26
Figure S.1: Sample UI – login page (light theme)....32
Figure S.2: Sample UI – seeker's applications with match scores and status....32
Figure S.3: MongoDB database resume_portal – collections and document counts....33
Figure S.4: Stored resume document with AI analysis (resumes collection)....33

LIST OF TABLES

Table 2.1: Comparison of existing systems with HireSense....3
Table 3.1: MongoDB collections....9
Table 3.2: Tools and technologies....9
Table 4.1: Main REST API endpoints (all prefixed with /api)....13
Table 4.2: Sample inputs and outputs....24
Table 4.3: Cross-browser and responsiveness checks....26
Table 4.4: Functional and validation test results....27

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Online recruitment has become the default channel through which students and professionals find employment. Large portals list thousands of openings, and employers increasingly rely on Applicant Tracking Systems (ATS) that parse resumes and filter them by keywords before a human ever reads them. As a result, a well-qualified candidate can be rejected simply because the resume uses different terminology, lacks quantified achievements, or is formatted in a way the parser cannot read.

At the same time, recent large language models (LLMs) can read unstructured text and produce structured, explainable judgements. Combined with classical information-retrieval techniques such as TF-IDF similarity, they make it possible to give job seekers recruiter-quality feedback within seconds and to give recruiters a ranked, explained shortlist instead of a pile of PDFs.

HireSense was developed as the Web Technology (23CS2048) project to apply the full web stack — HTML5, CSS3, JavaScript, React, Node.js, Express and MongoDB — to this real-world problem, and to integrate an AI service in a way that remains reliable when that service is slow or unavailable.

1.2 Problem Statement

Job seekers do not know how an ATS or a recruiter will judge their resume, which of their skills are being recognised, or which open jobs genuinely fit their profile. Recruiters, on the other hand, spend most of their screening time reading resumes that do not meet the basic requirements, and existing portals rarely explain *why* a candidate matches a job. Paid AI resume checkers exist, but they are separate from the job board, so feedback and matching are not connected. There is a need for a single, secure web portal in which resumes are analysed automatically, jobs and candidates are ranked by an explainable match score, and every role (seeker, recruiter and administrator) works on the same live data.

1.3 Objectives

- To design and develop a responsive, role-based web portal for job seekers, recruiters and administrators using React on the client and Node.js/Express on the server.

- To extract text from uploaded PDF, DOCX and TXT resumes and generate a structured AI analysis: ATS score, skills, experience, education, strengths, weaknesses, suggestions and suitable roles.
- To rank every open job for a seeker, and every applicant for a recruiter, using a hybrid score that combines required-skill overlap, TF-IDF text similarity and an LLM fit judgement with reasoning.
- To store users, resumes (including the original file), jobs and applications in MongoDB with validated Mongoose schemas and appropriate indexes.
- To secure the application with JWT authentication, bcrypt password hashing, role-based authorisation, and both client-side and server-side input validation.
- To guarantee availability through a rule-based fallback engine whenever the AI provider fails, and to deploy the system to the cloud.

1.4 Scope of the Project

The scope covers the complete hiring loop inside one portal: registration and login, resume upload and analysis, job posting and editing, AI-ranked job matches, applications with cover notes, applicant ranking, candidate discovery, application status tracking (applied, shortlisted, rejected, hired), and administrative moderation with dashboard statistics. Resumes are limited to text-based PDF, DOCX and TXT files of up to 4 MB; scanned image-only resumes, video interviews, payment features and e-mail notifications are outside the current scope. The AI provider is configurable: the hosted Groq API is used by default, and a fully local Ollama model can be selected when resume data must not leave the machine.

CHAPTER 2: LITERATURE SURVEY

2.1 Review of Existing Websites / Web Applications

LinkedIn Jobs. LinkedIn combines a professional profile with a job board. Candidates apply with their profile or an uploaded resume, and the platform shows how a member's listed skills compare with a posting. Matching is driven by the profile fields the user maintains rather than by an analysis of the resume document itself, and detailed insights are part of paid plans.

Naukri.com. Naukri is one of the largest Indian job portals. Recruiters search a large resume database with keyword and filter queries, and job seekers receive job recommendations based on their profile. Resume quality feedback is offered mainly as a separate paid resume-writing service.

Indeed. Indeed aggregates postings from many sources and lets employers screen applicants with questions and filters. Its strength is reach and search; it does not give candidates an explanation of how well their resume fits a specific job.

Jobscan and similar ATS resume checkers (e.g. Resume Worded). These tools let a user paste a resume and a job description to receive a keyword "match rate" and formatting tips. They provide useful feedback, but they are standalone utilities: the user must copy each job description manually, and recruiters do not see the results.

Commercial ATS products (e.g. Workday, Greenhouse). Enterprise ATS platforms parse resumes into structured fields and let recruiters filter applicants. They are designed for employers, are expensive, and largely rely on keyword rules that are opaque to candidates.

Research foundations. Classical information retrieval represents documents as TF-IDF weighted term vectors and compares them with cosine similarity [9], [10]. Transformer-based language models [11], [12] and open-weight models such as gpt-oss [13] can now return structured JSON judgements from free text, enabling explainable scoring that pure keyword matching cannot provide.

Table 2.1: Comparison of existing systems with HireSense

System	Resume analysis	Job/candidate ranking	Explanation of match	Recruiter + seeker in one place
LinkedIn Jobs	Profile-based	Yes	Partial (premium)	Yes
Naukri.com	Paid service	Keyword search	No	Yes
Indeed	No	Search / filters	No	Yes

System	Resume analysis	Job/candidate ranking	Explanation of match	Recruiter + seeker in one place
Jobscan / Resume Worded	Yes (keywords)	One job at a time	Keyword list	No (seeker only)
Enterprise ATS	Parsing only	Rule-based filters	No	No (recruiter only)
HireSense	LLM + rules	All jobs / all candidates	AI reasoning + missing skills	Yes, plus admin

2.2 Summary of Existing Approaches

- **Keyword and filter search:** recruiters query a resume database by keywords, location and experience. Fast, but brittle with synonyms (e.g. “Node” vs “Node.js”) and gives no ranking explanation.
- **Profile-field matching:** platforms compare structured profile fields with job fields. Accurate only when users keep profiles complete and current.
- **ATS parsing with rules:** resumes are parsed into sections and scored with fixed rules, which reward formatting and exact keywords over real suitability.
- **Standalone AI resume checkers:** provide document feedback against one pasted job description, disconnected from live job listings and from recruiters.

2.3 Research Gap or Limitations of Existing Systems

- Resume feedback and job matching are separate products; a candidate cannot see, in one place, how their resume is judged and which live jobs it fits.
- Match scores are rarely explained. Candidates do not learn which required skills are missing, and recruiters cannot see why a candidate ranks highly.
- Pure keyword systems fail on synonyms and context, while pure LLM systems are slow, can return malformed output and become unavailable when the provider is down.
- Recruiters only see candidates who applied; suitable candidates already on the platform are not surfaced.
- Most AI features require paid plans, which limits access for students and freshers.

HireSense addresses these gaps by combining a deterministic, instant hybrid score with an on-demand LLM judgement, exposing the matched and missing skills and the AI reasoning to both sides, adding a “Discover candidates” view for recruiters, and falling back to rules whenever the AI cannot answer.

CHAPTER 3: METHODOLOGY AND SYSTEM DESIGN

3.1 Proposed Methodology

The project followed an iterative, incremental model. Requirements were first captured as user stories for the three roles; the database schema and REST API were then designed, followed by the AI pipeline, the React interface and finally deployment and testing. Each iteration delivered a working vertical slice (for example, “upload a resume and see its analysis”) that was tested end to end before the next feature was started. Work was tracked in a shared GitHub repository.

The core of the methodology is the resume-processing and matching pipeline shown in Figure 3.1. The uploaded file is converted to plain text, analysed by the LLM in JSON mode, validated and blended with rule-based signals, and stored. Matching uses two levels: a fast deterministic score computed for every open job on each request, and a slower AI fit score requested only when the user applies, checks fit or re-scores.

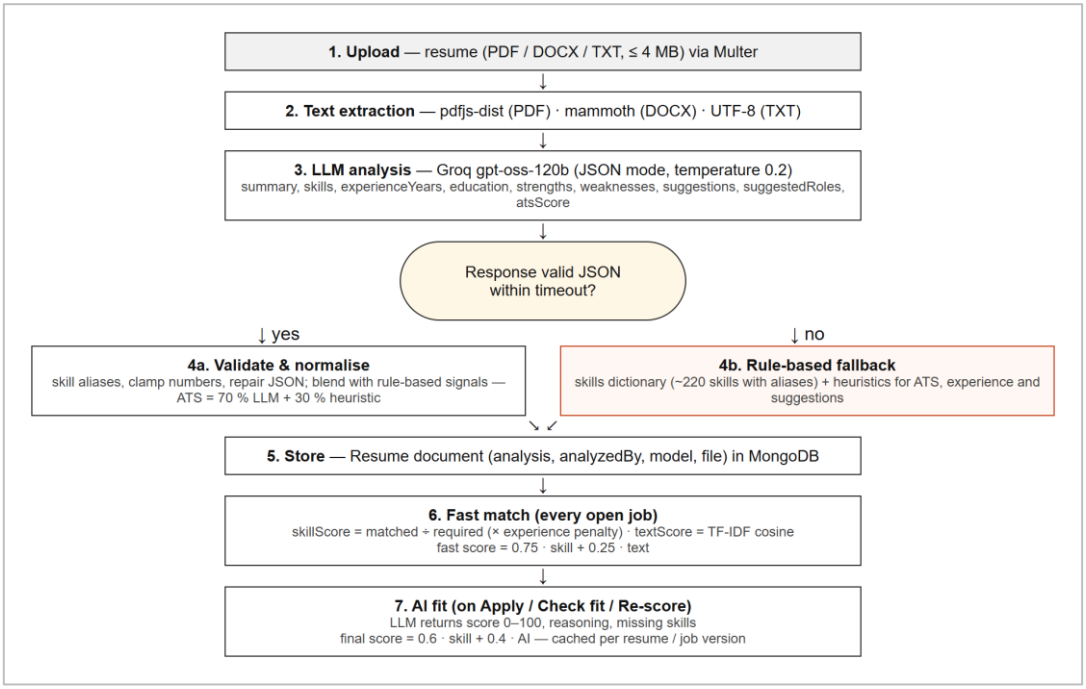


Figure 3.1: Resume analysis and job matching pipeline

The scoring formulas are: **skillScore** = matched required skills ÷ required skills × 100 (reduced proportionally when the candidate has less than the minimum experience); **textScore** = TF-IDF cosine similarity between the resume and the job text, scaled to 0–100; **fast score** = 0.75 × skillScore + 0.25 × textScore; and, once the LLM has judged the fit, **final score** = 0.6 × skillScore + 0.4 × AI score.

3.2 System Architecture

HireSense uses a three-tier client–server architecture (Figure 3.2). The **presentation tier** is a React single-page application that runs in the browser and communicates only through HTTPS JSON requests. The **application tier** is an Express 5 REST API that applies security middleware, authentication, role checks and validation before calling controllers and services; the services layer contains the text extractor, the resume analyser, the matcher, the LLM client and the rule-based fallback. The **data tier** is MongoDB, accessed through Mongoose. The external AI provider (Groq or Ollama) is called only by the application tier, so API keys never reach the browser.

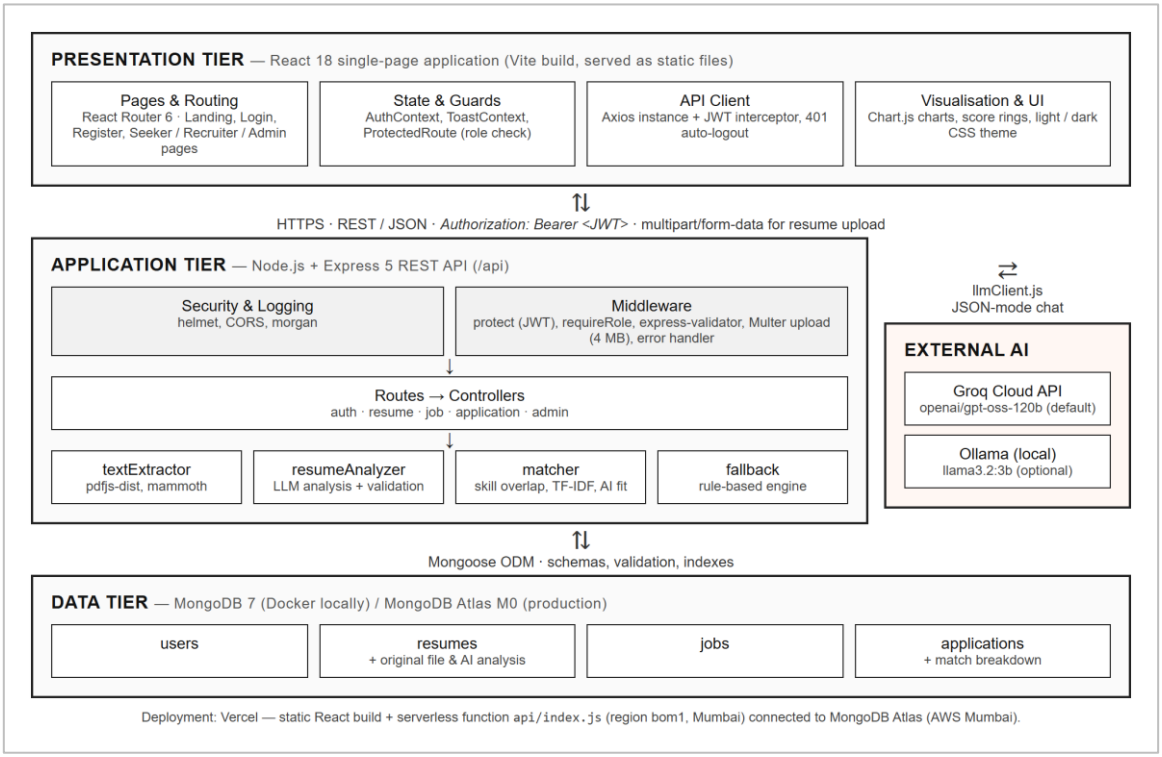


Figure 3.2: Three-tier system architecture of HireSense

3.3 Website Structure and Navigation Flow

Figure 3.3 shows the sitemap. Public visitors see the landing, login and registration pages. After authentication the server returns a JWT, the AuthContext stores the session, and the ProtectedRoute component sends each user to the home page of their role. Every signed-in page shares a navigation bar with a role-specific menu, an AI status badge (AI online / Model missing / AI fallback), a theme toggle and a log-out action.

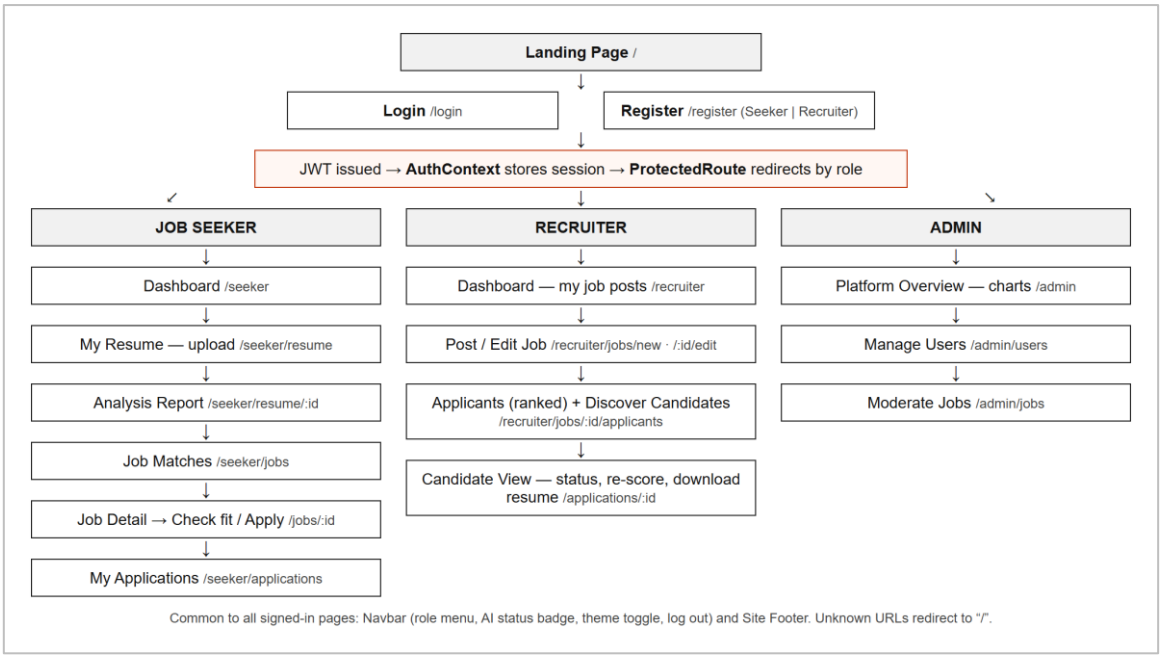


Figure 3.3: Sitemap and navigation flow

3.4 UML Diagram – Use Case Diagram

The use case diagram (Figure 3.4) identifies three actors. The Job Seeker uploads and analyses resumes, views the ATS report, browses ranked job matches, checks AI fit, applies and tracks applications. The Recruiter manages job posts, views ranked applicants, discovers matching candidates, changes application status, re-scores applicants and downloads resumes. The Admin views platform statistics and manages users and job posts. All actors authenticate through the shared Register / Log in use case; use cases marked «AI» are served by the LLM with the rule-based fallback.

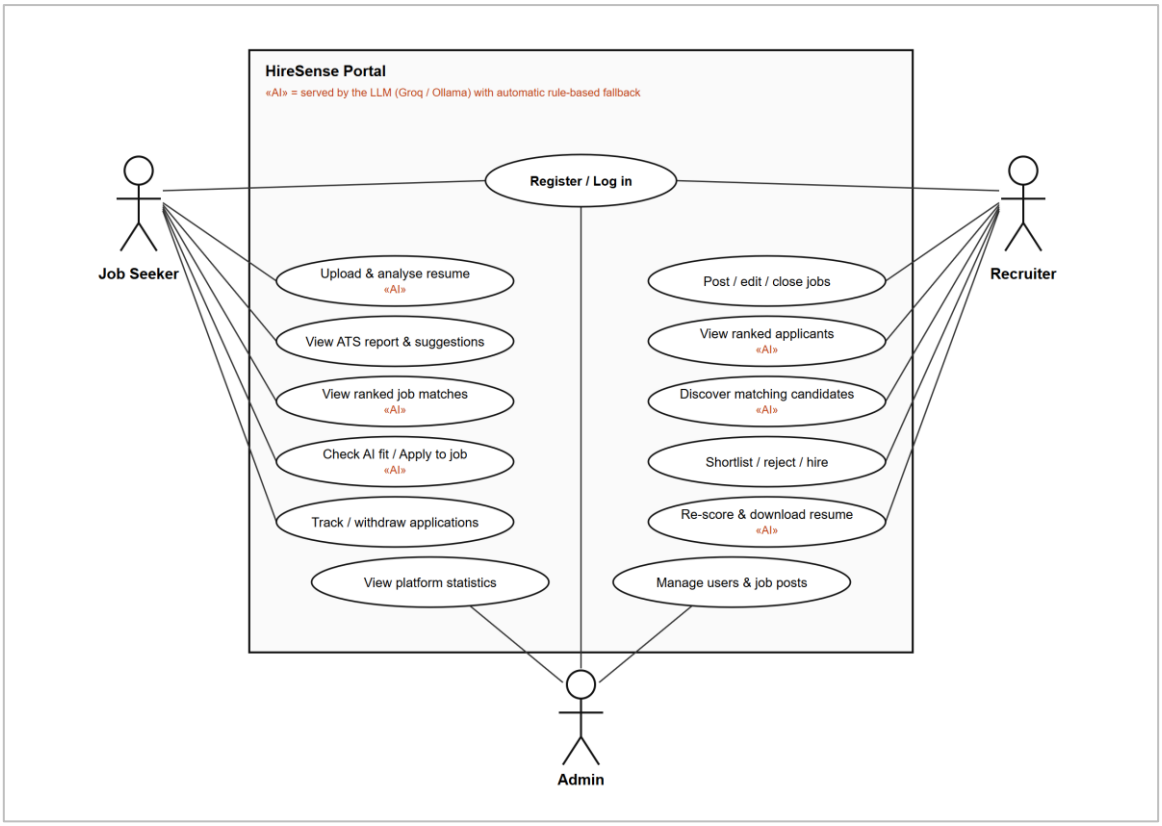


Figure 3.4: Use case diagram

3.5 Database Design

MongoDB was chosen because resume analyses and match breakdowns are naturally nested documents whose fields can evolve, while the main entities still relate to each other through references. Four collections are used; their relationships are shown in the ER diagram in Figure 3.5 and summarised in Table 3.1.

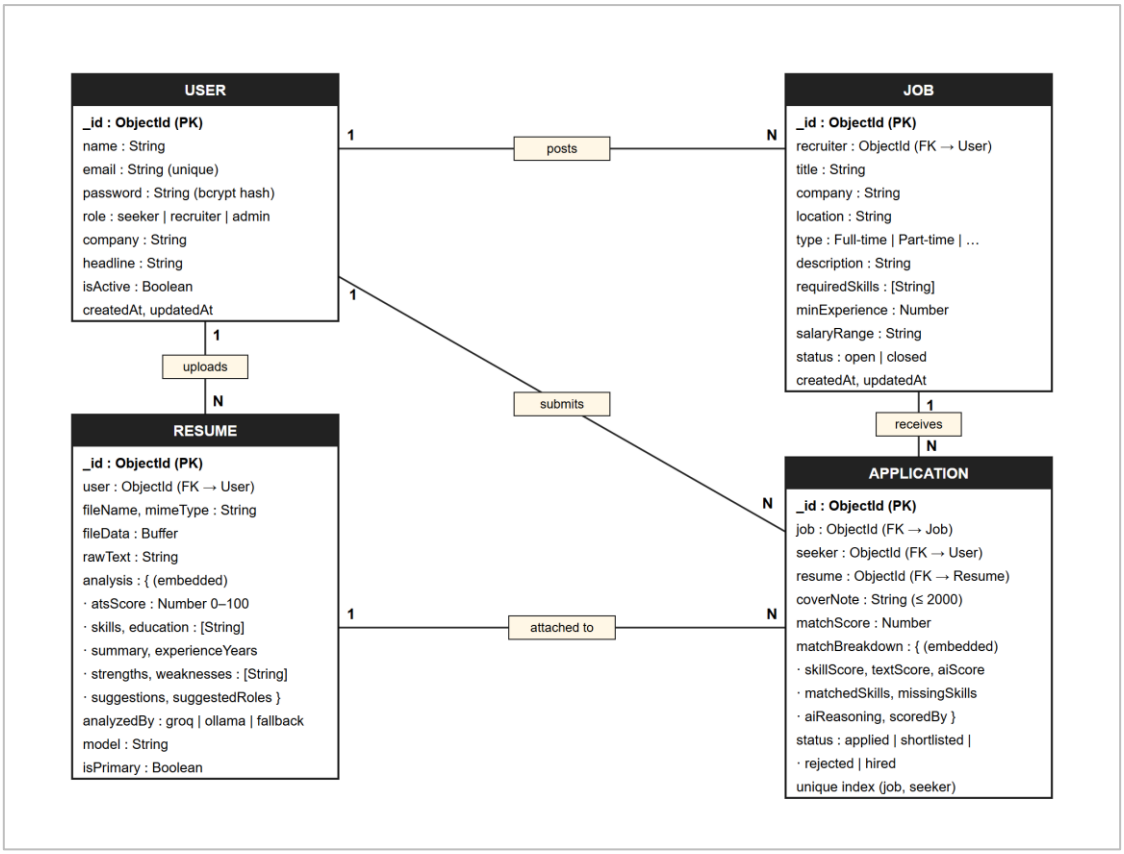


Figure 3.5: Entity–relationship diagram of the MongoDB collections

Table 3.1: MongoDB collections

Collection	Purpose	Key fields / constraints
users	Seekers, recruiters and admins	email unique and lower-cased; password bcrypt-hashed and excluded from queries; role enum; isActive flag
resumes	Uploaded resumes and AI analysis	user reference; original file stored as Buffer (≤ 4 MB); rawText; embedded analysis sub-document; analyzedBy (groq / ollama / fallback); isPrimary
jobs	Job posts created by recruiters	recruiter reference; requiredSkills array; minExperience ≥ 0; type and status enums; text index on title, description and company
applications	A seeker's application to a job	job, seeker and resume references; matchScore; embedded matchBreakdown; status enum; unique compound index (job, seeker) prevents duplicate applications

3.6 Tools and Technologies Used

Table 3.2: Tools and technologies

Layer	Technology	Use in the project
Front end	HTML5, CSS3, JavaScript (ES2022)	Semantic markup, custom responsive CSS with light/dark themes, client logic
Front end	React 18, React Router 6, Vite 6	Component-based SPA, client-side routing with role guards, fast build and dev server

Layer	Technology	Use in the project
Front end	Axios, Chart.js (react-chartjs-2)	HTTP client with JWT interceptor; admin dashboard charts
Back end	Node.js 20+, Express 5	REST API server, routing and middleware
Back end	jsonwebtoken, bcryptjs, helmet, cors, morgan	Authentication, password hashing, security headers, CORS policy, request logging
Back end	express-validator, Multer	Server-side input validation; multipart upload with type and size limits
Back end	pdfjs-dist, mammoth	Text extraction from PDF and DOCX resumes
AI	Groq API (openai/gpt-oss-120b), Ollama (llama3.2:3b)	Resume analysis and job-fit judgement in JSON mode
Database	MongoDB 7, Mongoose 8	Document storage, schemas, validation and indexes
Other tools	VS Code, Git / GitHub, Docker, Postman, Chrome DevTools	Development, version control, local MongoDB container, API testing, debugging
Deployment	Vercel, MongoDB Atlas	Static hosting + serverless API (Mumbai region); managed cloud database

3.7 Module Description

3.7.1 Authentication and User Management Module

Handles registration (as seeker or recruiter; recruiters must give a company name), login, profile update and session restore. Passwords are hashed with bcrypt in a Mongoose pre-save hook; a signed JWT carrying the user id and role is returned on login and verified by the `protect` middleware on every protected route. The `requireRole` middleware restricts routes to specific roles, and deactivated accounts are rejected.

3.7.2 Resume Upload and AI Analysis Module

Accepts PDF, DOCX or TXT files up to 4 MB through Multer, extracts the text, and produces the analysis described in Section 3.1. Seekers can keep several resumes, mark one as primary for matching, re-run the analysis, and delete resumes that are not attached to applications. The analysis report page shows the ATS score ring, skills, experience, education, AI-flagged issues with suggested fixes, strengths and suggested roles.

3.7.3 Job Management Module

Recruiters create, edit, close and delete job posts with title, company, location, type, salary range, minimum experience, description and required skills (entered with an auto-complete skill input). All signed-in users can search open jobs by keyword, location, type and skills.

3.7.4 Matching and Application Module

Ranks all open jobs for a seeker's primary resume, provides the AI "Check my fit" explanation, and records applications with a cover note and a stored score breakdown. Recruiters see applicants ranked by score, can filter by status, change status, re-score with the AI, open a full candidate view and download the original resume. The "Discover candidates" tab ranks every seeker on the platform against a job.

3.7.5 Administration Module

Gives administrators an overview dashboard (users by role, jobs, applications, analysed resumes, sign-ups over the last 14 days, applications by status, top candidate skills and most demanded skills) and pages to activate/deactivate users, change roles, delete users, and close or delete job posts.

3.7.6 AI Provider and Fallback Module

`llmClient.js` wraps the Groq chat-completions API and the Ollama chat API behind one `chatJSON` function with timeouts, one retry on rate-limit, and tolerant JSON parsing. A cached health check powers the AI status badge. When the provider is not ready or the output is invalid, `fallback.js` produces the analysis with the skills dictionary and heuristic ATS checks, and the result is labelled "fallback" so users know which engine produced it.

CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1 Implementation Details

The repository is organised as a monorepo with a `client` folder (React application), a `server` folder (Express API) and an `api/index.js` entry point that runs the same Express app as a Vercel serverless function. During development `npm run dev` starts both the API (port 5000) and the Vite dev server (port 5173), and MongoDB runs in a Docker container defined in `docker-compose.yml`. A seed script creates demo users, 20 jobs, 5 analysed resumes and 13 applications.

4.1.1 Front-End Development

UI design. The interface follows a "blueprint paper" visual language: a warm paper background with a hairline grid, large uppercase headlines, monospace section labels such as [01], square cards without rounded corners, and a single orange accent. All colours are CSS custom properties, which allows a light and a dark theme that follows the operating-system preference or the user's toggle.

Components and routing. Pages are composed from reusable components — Navbar, JobCard, ScoreRing, MatchBreakdown, SkillInput, Charts, ApplyModal and ProtectedRoute. React Router defines public routes and role-guarded route groups; unknown URLs redirect to the home page. Global state is limited to two React contexts: AuthContext (current user and token) and ToastContext (notifications).

Responsive layout. Layouts use CSS Grid and Flexbox with media queries; at phone widths the navigation collapses into a menu button and multi-column grids stack into a single column (see Figure 4.12).

Form validation. Forms use HTML5 constraints (`required`, `type="email"`, `minLength`) for immediate feedback, plus JavaScript checks before submission — for example, a job cannot be published without at least one required skill. Errors returned by the server are displayed in a form-level error banner.

```
// client/src/api/client.js – every request carries the JWT; an expired session logs the
user out
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

4.1.2 Back-End Development

The Express application applies Helmet, CORS (restricted to the client URL), JSON body parsing and Morgan logging, then mounts all routes under `/api`. Each route is a chain of middleware: authentication (`protect`), authorisation (`requireRole`), validation rules, and finally the controller. A central error handler converts validation errors, Multer errors, Mongoose cast/validation errors and duplicate-key errors into consistent JSON responses with the correct HTTP status code. Table 4.1 lists the main endpoints.

```
// server/src/routes/index.js – registration with server-side validation
router.post('/auth/register',
  body('name').trim().notEmpty().withMessage('Name is required'),
  body('email').isEmail().withMessage('A valid email is required'),
  body('password').isLength({ min: 6 }).withMessage('Password must be at least 6
characters'),
  validate, auth.register);

// role guard reused across route groups
const recruiter = [protect, requireRole('recruiter', 'admin')];
router.post('/jobs', ...recruiter, ...jobValidators, jobs.createJob);
```

Table 4.1: Main REST API endpoints (all prefixed with /api)

Method	Endpoint	Access	Description
POST	/auth/register, /auth/login	Public	Create account / log in and receive a JWT
GET	/auth/me	Any user	Current user profile
GET	/ai/health	Public	AI provider status for the navbar badge
POST	/resumes	Seeker	Upload (multipart) and analyse a resume
POST	/resumes/:id/reanalyze	Seeker	Run the analysis again
GET	/jobs, /jobs/:id	Any user	Search open jobs / job detail
GET	/jobs/matches	Seeker	Open jobs ranked for the primary resume
GET	/jobs/:id/fit	Seeker	AI fit check with reasoning
POST/PUT/DELETE	/jobs, /jobs/:id	Recruiter	Create, update, delete job posts
GET	/jobs/:id/candidates	Recruiter	Rank all seekers for a job
POST	/applications	Seeker	Apply to a job (computes AI match score)
GET	/applications/job/:jobId	Recruiter	Applicants ranked by score
PATCH	/applications/:id/status	Recruiter	Shortlist / reject / hire
POST	/applications/:id/rescore	Recruiter	Re-run AI scoring
GET	/admin/stats	Admin	Dashboard statistics
GET/PATCH/DELETE	/admin/users, /admin/jobs	Admin	Manage users and moderate jobs

The AI integration sends a strict system prompt that asks for a JSON object with fixed keys and an explicit ATS rubric (section headings, contact information, keywords, quantified

achievements, date consistency, action verbs and education). The response is validated, numbers are clamped, skill names are normalised through an alias dictionary, and the ATS score is blended with the rule-based score so that it is stable between runs.

4.1.3 Database Connectivity

The server connects to MongoDB with Mongoose using the `MONGO_URI` environment variable (a Docker container locally and MongoDB Atlas in production). Because serverless functions start and stop frequently, `ensureDB()` creates the connection once, reuses it across invocations, and retries on the next request if a connection attempt fails; the serverless entry point returns HTTP 503 with a JSON message instead of crashing when the database is unreachable.

```
// server/src/config/db.js – connect once and reuse across serverless invocations
export async function ensureDB() {
  if (mongoose.connection.readyState === 1) return;
  connecting ??= connectDB().catch((err) => { connecting = undefined; throw err; });
  await connecting;
}
```

Schemas enforce data integrity at the database layer: required fields, enums for roles, job types and statuses, numeric ranges, a unique e-mail index, a text index for job search and a unique compound index that prevents a seeker from applying twice to the same job. Sensitive fields (`password`, `fileData`, `rawText`) use `select: false` so they are never returned unless explicitly requested. Figures 4.1–4.3 show documents stored by the application, queried with mongosh.

```
mongosh — mongodb://127.0.0.1:27017/resume_portal

resume_portal> db.users.find({}, {name:1,email:1,role:1,company:1,password:1}).limit(2)
[
  {
    _id: ObjectId('6ae216012697c3b59519c5e2'),
    name: 'Portal Admin',
    email: 'admin@demo.com',
    password: '$2a$10$lUi8BNP.m0mwYzPNNPV46u1G9bmcsH3dNRLFCkXyFES/wa1H6irvG',
    role: 'admin',
    company: ''
  },
  {
    _id: ObjectId('6ae216012697c3b59519c5e6'),
    name: 'Kavya Nair',
    email: 'recruiter@demo.com',
    password: '$2a$10$hhExOhQBDiqwt8kXYyiSUuTcgQ9Cn4G/xRLqut66096LuPaT7h7Xa',
    role: 'recruiter',
    company: 'TechCorp Solutions'
  }
]
```

Figure 4.1: users collection – passwords stored only as bcrypt hashes

```
mongosh — mongodb://127.0.0.1:27017/resume_portal

resume_portal> db.jobs.findOne({}, {description:0})
{
  _id: ObjectId('6ae216022697c3b59519c5fe'),
  recruiter: ObjectId('6ae216012697c3b59519c5e6'),
  title: 'MERN Stack Developer',
  company: 'TechCorp Solutions',
  location: 'Chennai',
  type: 'Full-time',
  requiredSkills: [
    'MongoDB',
    'Express.js',
    'React',
    'Node.js',
    'JavaScript',
    'REST API',
    'Git'
  ],
  minExperience: 1,
  salaryRange: '₹6-10 LPA',
  status: 'open',
  createdAt: ISODate('2026-10-28T15:34:26.370Z'),
  updatedAt: ISODate('2026-10-28T15:34:26.370Z'),
  __v: 0
}
```

Figure 4.2: jobs collection – a job post created by a recruiter

```
mongosh — mongodb://127.0.0.1:27017/resume_portal

resume_portal> db.applications.findOne({}, {coverNote:0})
{
  _id: ObjectId('6ae216192697c3b59519c630'),
  job: ObjectId('6ae216022697c3b59519c5fe'),
  seeker: ObjectId('6ae216022697c3b59519c5ec'),
  resume: ObjectId('6ae216062697c3b59519c626'),
  matchScore: 93,
  matchBreakdown: {
    skillScore: 100,
    textScore: 73,
    aiScore: null,
    matchedSkills: [
      'MongoDB',
      'Express.js',
      'React',
      'Node.js',
      'JavaScript',
      'REST API',
      'Git'
    ],
    missingSkills: [],
    aiReasoning: '',
    scoredBy: 'fallback'
  },
  status: 'shortlisted',
  createdAt: ISODate('2026-10-28T15:34:49.121Z'),
  updatedAt: ISODate('2026-10-28T15:34:49.121Z'),
  __v: 0
}
```

Figure 4.3: applications collection – match score and embedded score breakdown

4.2 Screenshots of the Web Application

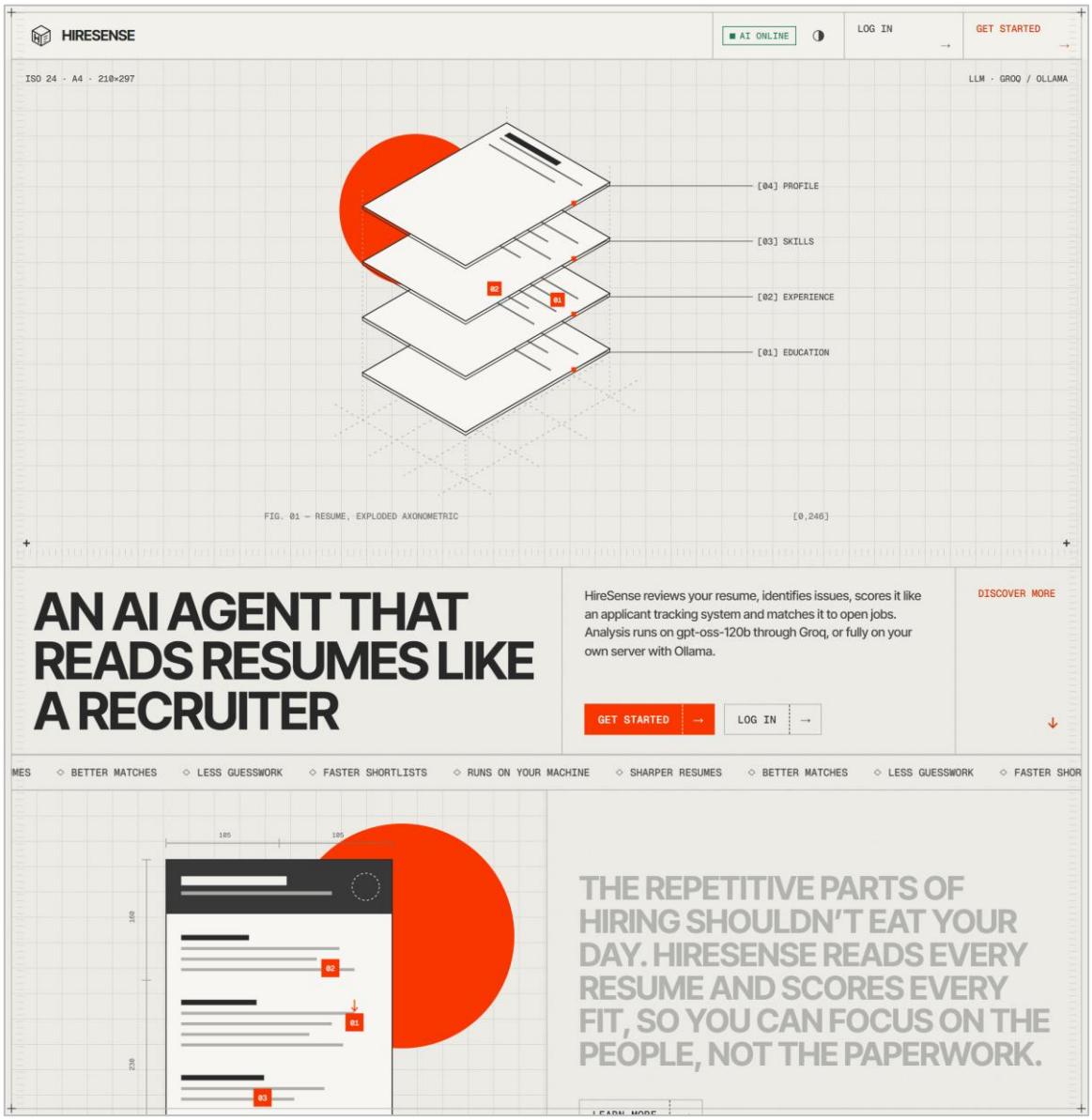


Figure 4.4: Landing page

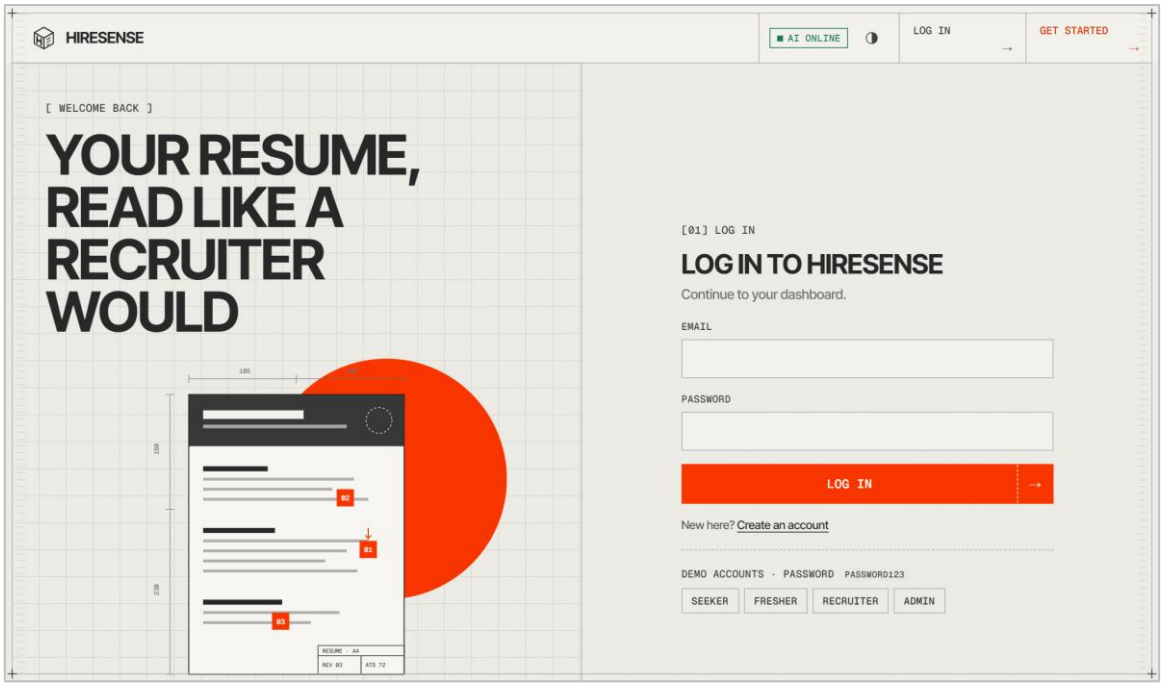


Figure 4.5: Login page

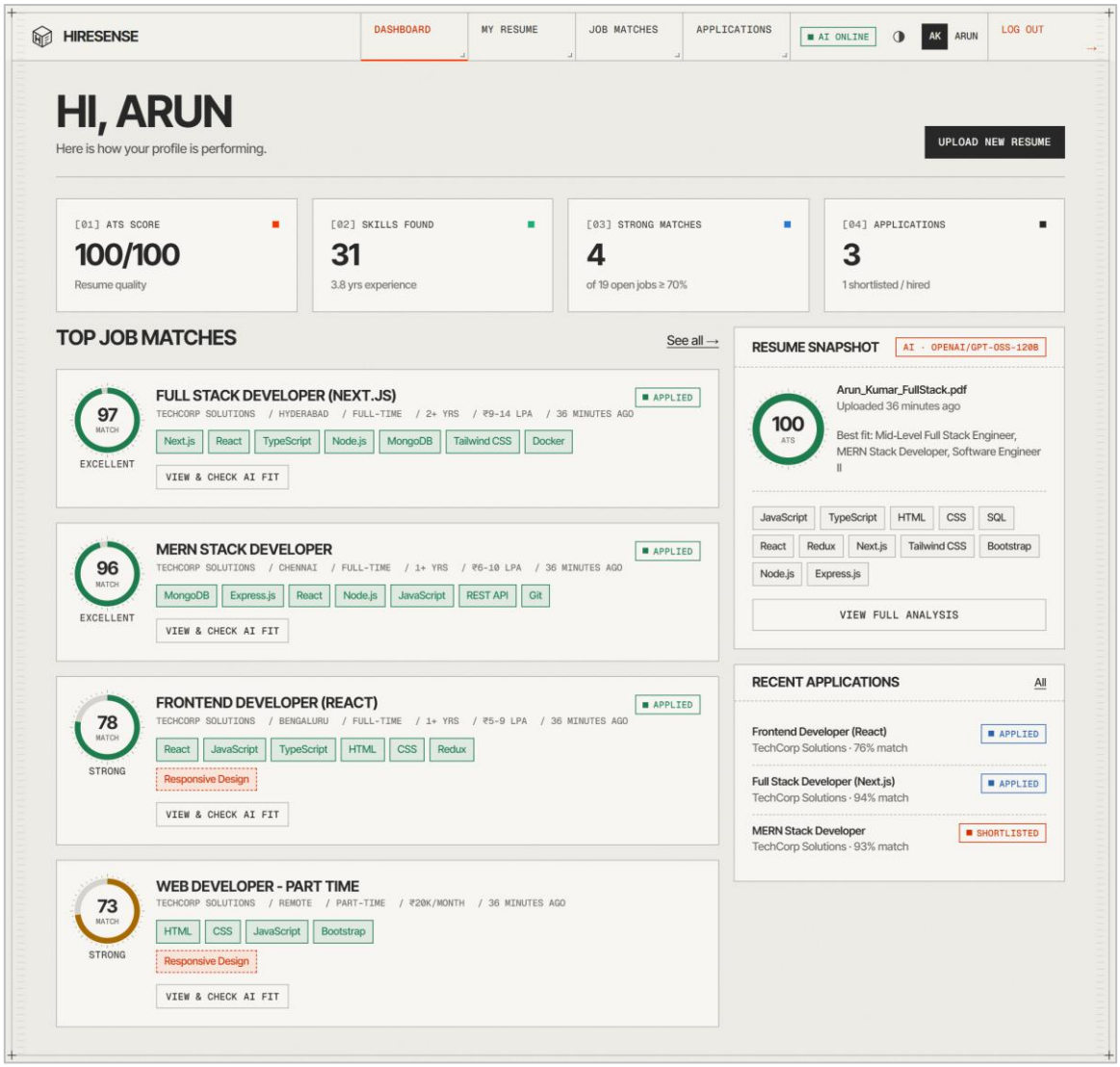


Figure 4.6: Job seeker dashboard with top job matches and resume snapshot

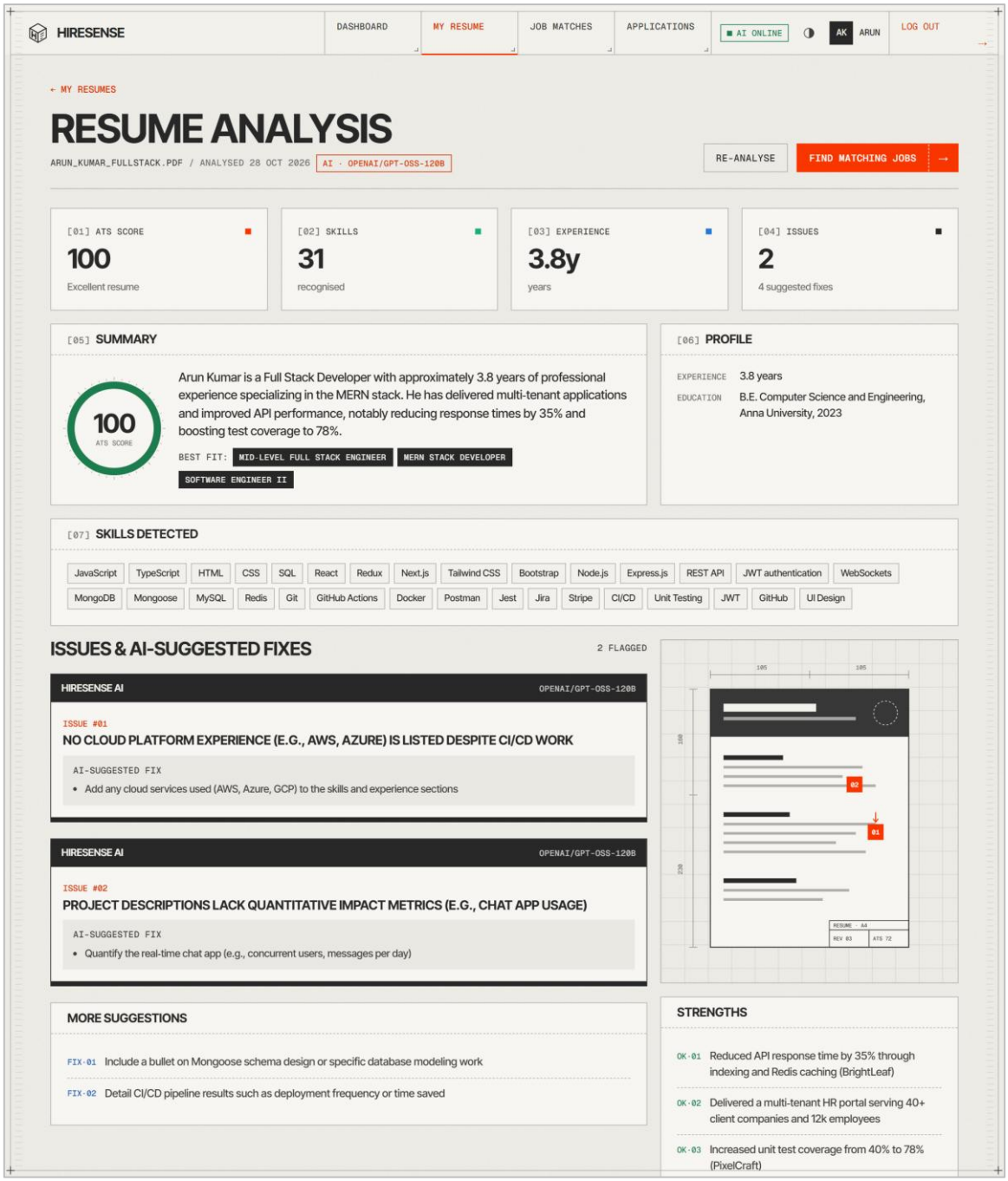


Figure 4.7: AI resume analysis report – ATS score, skills, issues and suggested fixes

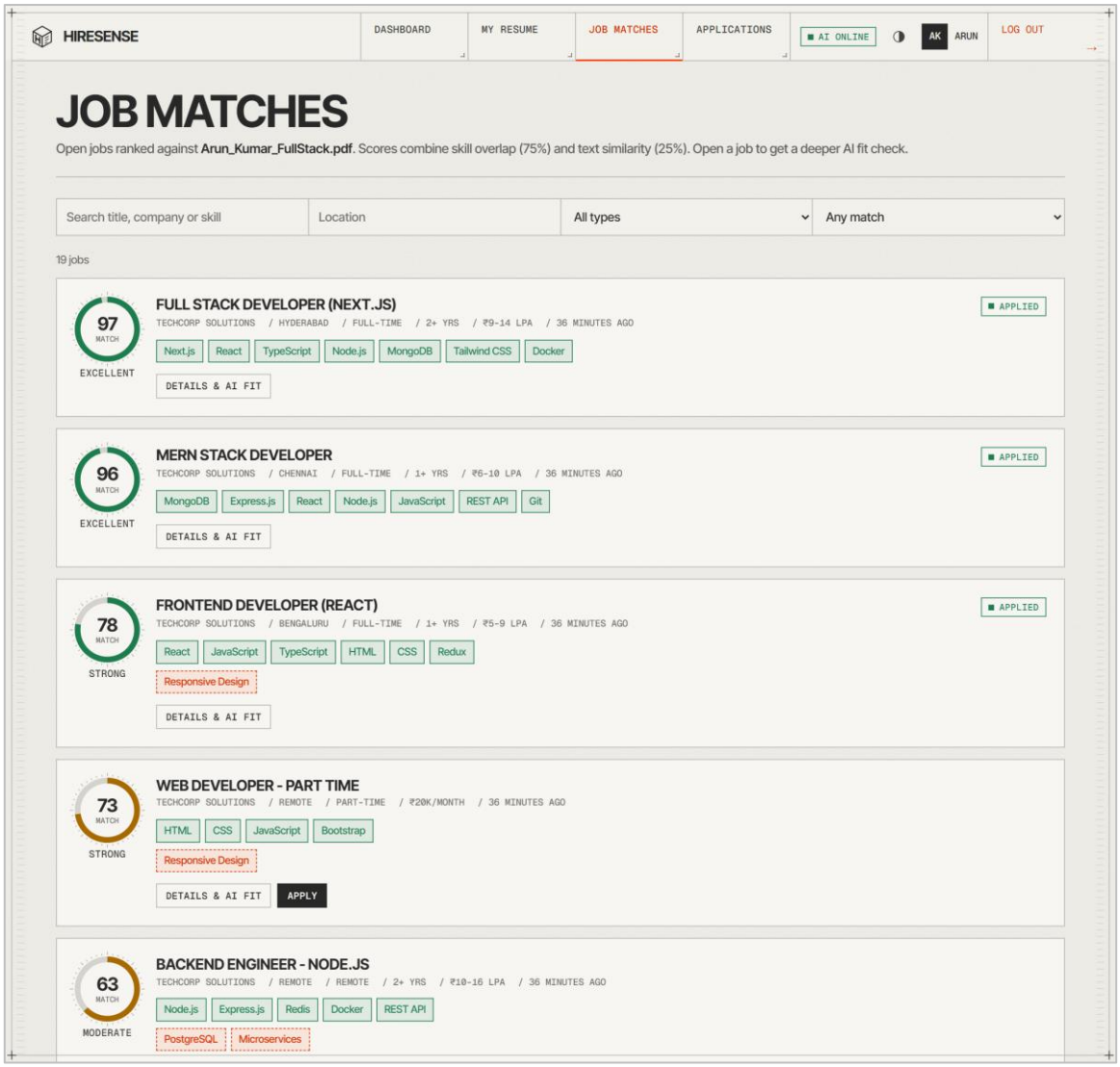


Figure 4.8: Job matches ranked by match score

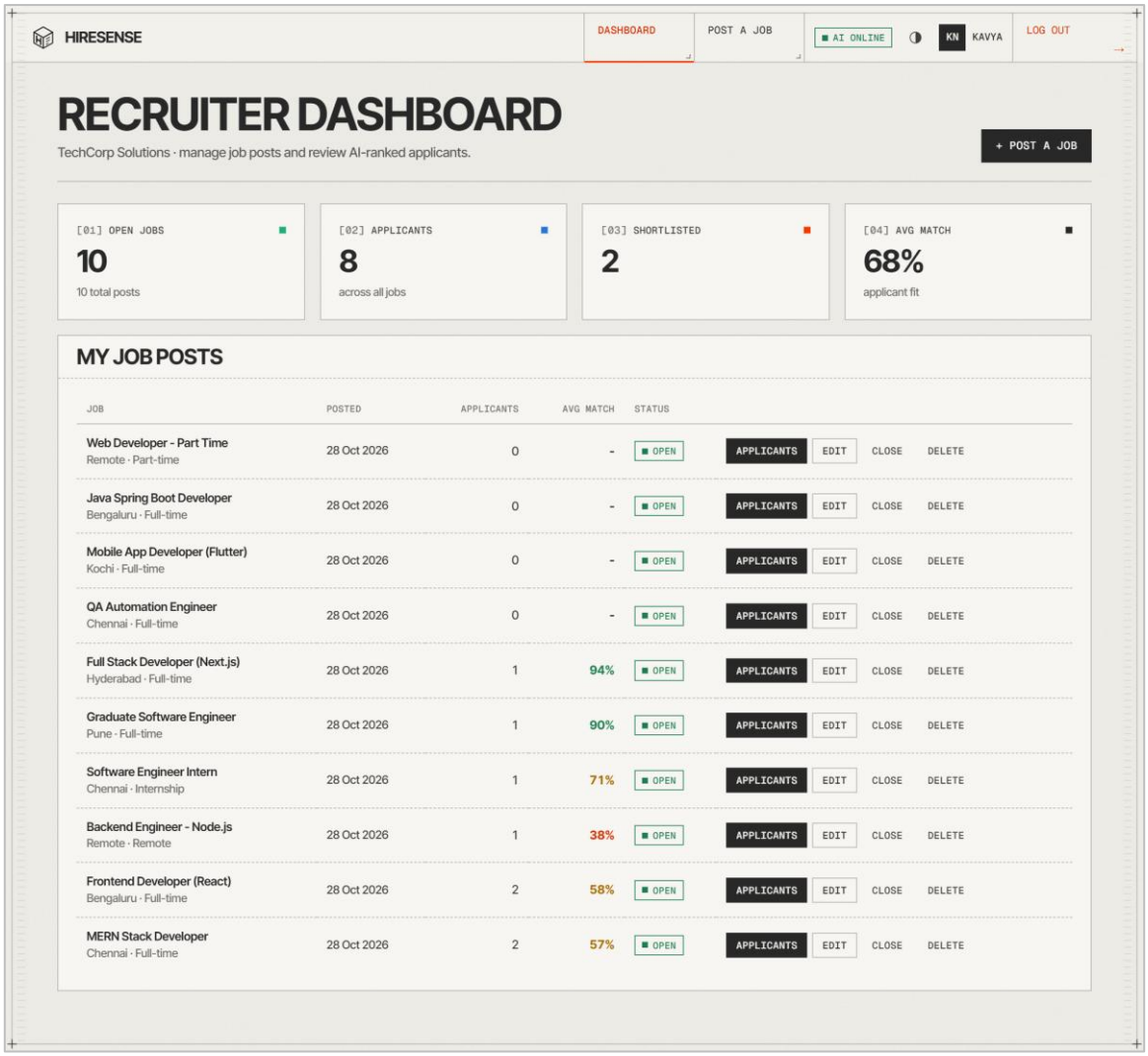


Figure 4.9: Recruiter dashboard – job posts with applicant statistics

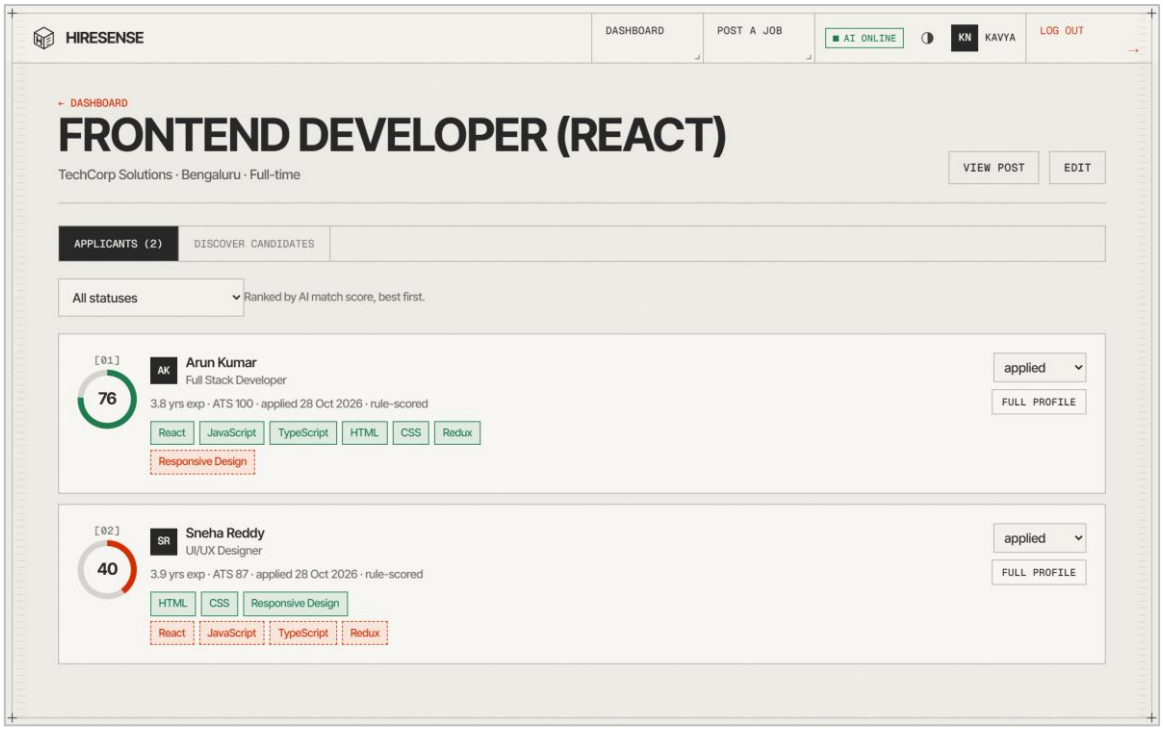


Figure 4.10: Applicants for a job ranked by AI match score



Figure 4.11: Admin platform overview with charts

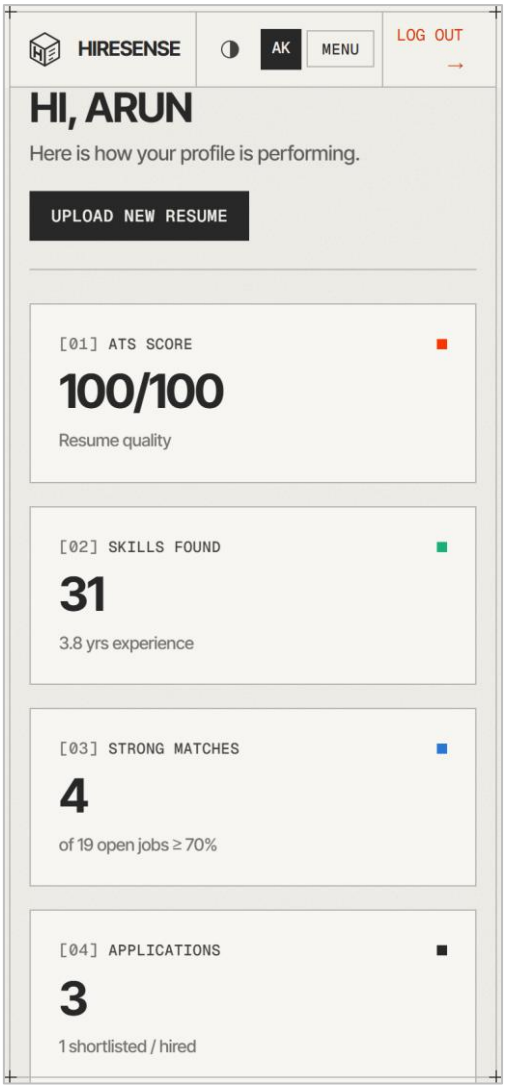


Figure 4.12: Responsive layout on a mobile viewport (390 px wide)

4.3 Sample Inputs and Outputs

Table 4.2: Sample inputs and outputs

Input	Output produced by the system
Seeker uploads *Arun_Kumar_FullStack.pdf* (full-stack developer resume)	ATS score 100/100, 31 skills recognised (JavaScript, TypeScript, React, Node.js, MongoDB, Docker …), 3.8 years of experience, suggested roles "Mid-level Full Stack Engineer, MERN Stack Developer, Software Engineer II", 2 AI-flagged issues with fixes (Figure 4.7)
Fresher uploads *Rahul_Verma_Fresher.pdf*	Analysed by Groq in ≈3.6 s: ATS score 85, 15 skills detected; 19 open jobs ranked for the resume
Arun applies to "MERN Stack Developer" (TechCorp)	Application stored with matchScore 93: skillScore 100 (7/7 required skills matched), textScore 73, status later set to "shortlisted" (Figure 4.3)
Recruiter opens applicants for "Frontend Developer (React)"	Two applicants listed in descending order of match score with matched and missing skills (Figure 4.10)
Recruiter tries to publish a job without required skills	Form blocked with "Add at least one required skill – it is the main signal for AI matching."

Input	Output produced by the system
User registers with an e-mail that already exists	HTTP 409 and the banner “An account with this email already exists”

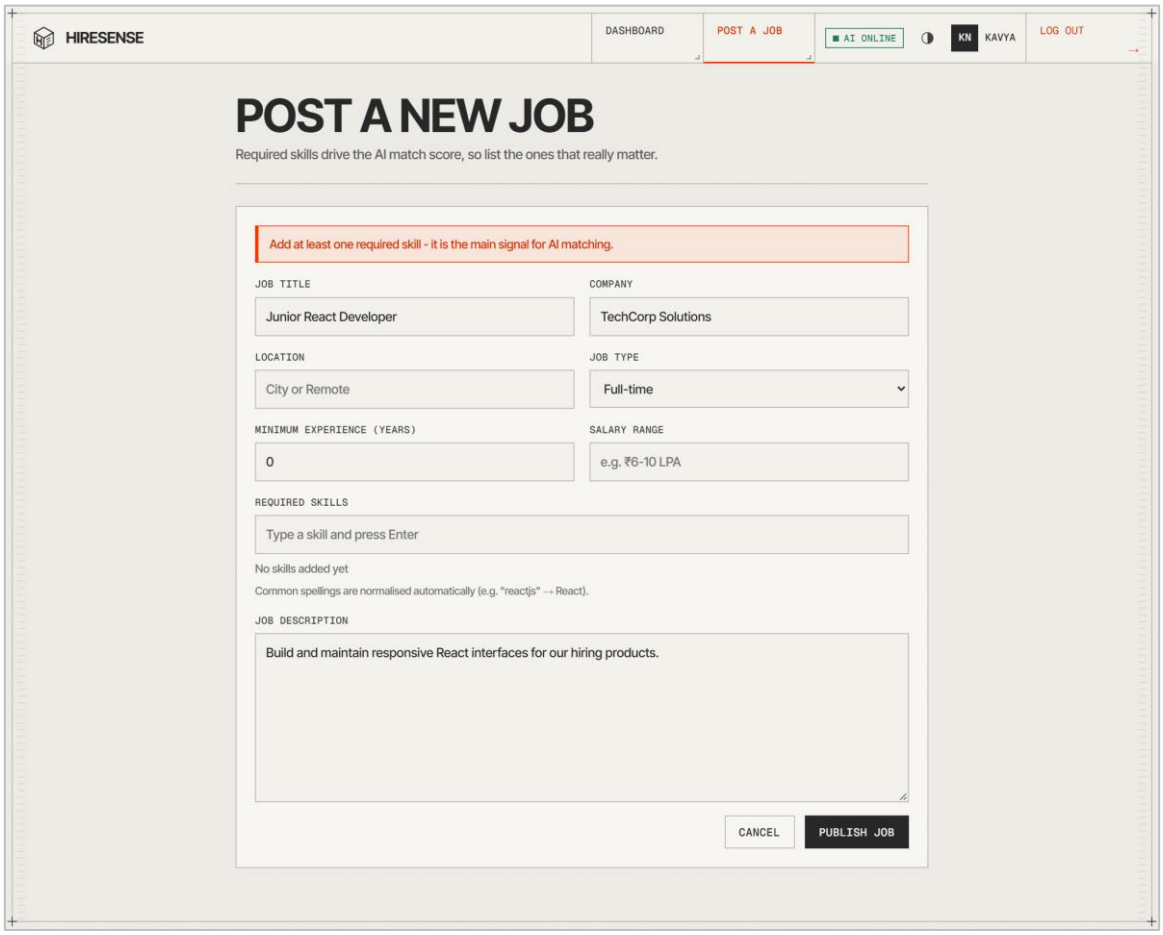


Figure 4.13: Post a job form showing a client-side validation message

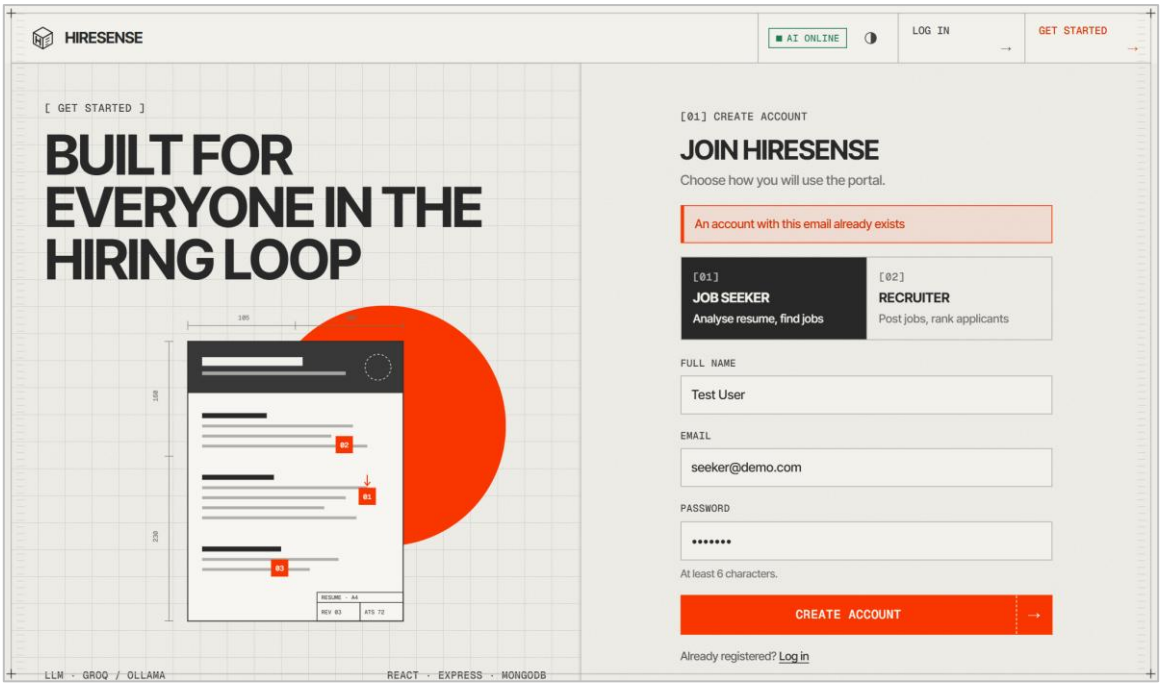


Figure 4.14: Registration form showing a server-side validation error (duplicate e-mail)

4.4 Testing and Results

4.4.1 Unit Testing

Each service function was tested in isolation during development with sample data: `extractText` with PDF, DOCX and TXT resumes from the `samples` folder; `analyzeWithRules` with resumes of different profiles (full-stack, data science, DevOps, UI/UX and fresher); `fastMatch` with jobs whose required skills were fully, partly and not matched, and with candidates below the minimum experience; and `parseJsonLoose` with model responses wrapped in markdown fences or containing trailing commas. The rule-based fallback was tested by stopping the AI provider and confirming that uploads still produced an analysis labelled “fallback”.

4.4.2 Cross-Browser and Responsiveness Testing

Table 4.3: Cross-browser and responsiveness checks

Environment	Viewport	Checks	Result
Google Chrome (desktop)	1366 × 800	All pages for the three roles, light and dark themes, charts, file upload	Pass
Chrome mobile emulation (Android)	390 × 844	Collapsed navigation menu, stacked cards and forms, no horizontal scroll	Pass
Tablet emulation	768 × 1024	Two-column grids, tables scroll inside their container	Pass
Production build on Vercel	Desktop and mobile	Routing of deep links (SPA rewrite), API calls to the serverless function	Pass

4.4.3 Form Validation and Functional Testing

A scripted functional test exercised the running API with valid and invalid requests. Table 4.4 lists every case with the expected and the observed HTTP status; all 23 cases produced the expected result. The valid resume upload, which calls the Groq LLM, completed in about 3.6 seconds, while all other requests completed within 65 ms on the local server.

Table 4.4: Functional and validation test results

#	Test case	Expected	Observed status / message	Result
1	Register – invalid email	400 + message	400 – A valid email is required	Pass
2	Register – short password	400 + message	400 – Password must be at least 6 characters	Pass
3	Register – empty name	400 + message	400 – Name is required	Pass
4	Register – duplicate email	409 Conflict	409 – An account with this email already exists	Pass
5	Login – wrong password	401	401 – Invalid email or password	Pass
6	Protected route without token	401	401 – Not authenticated	Pass
7	Tampered JWT	401	401 – Session expired, please log in again	Pass
8	Login seeker@demo.com	200 + token	200	Pass
9	Login recruiter@demo.com	200 + token	200	Pass
10	Login fresher@demo.com	200 + token	200	Pass
11	Seeker calls admin API	403	403 – This action requires role: admin	Pass
12	Seeker posts a job	403	403 – This action requires role: recruiter or admin	Pass
13	Job – description < 30 chars	400	400 – Description should be at least 30 characters	Pass
14	Job – experience out of range	400	400 – Experience must be 0-40 years	Pass
15	Apply – missing jobId	400	400 – jobId is required	Pass
16	Apply – duplicate application	409	409 – You have already applied to this job	Pass
17	Invalid ObjectId	400	400 – Invalid _id	Pass
18	Upload – .exe file	400	400 – Only PDF, DOCX or TXT resumes are allowed	Pass
19	Upload – 5 MB file	400	400 – File is too large (max 4 MB)	Pass
20	Upload – valid PDF resume	201 + analysis	201 – ATS 85, 15 skills, analysedBy=groq	Pass
21	Ranked job matches	200 + ranked list	200 – 19 jobs ranked	Pass
22	Delete test resume	200	200 – Resume deleted	Pass
23	Unknown route	404	404 – Route not found: GET /api/nope	Pass

CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENT

5.1 Summary of the Work

HireSense was designed and implemented as a complete MERN-stack web application with three roles. The work included the database design for four MongoDB collections, a secured REST API of more than thirty endpoints, an AI pipeline that analyses resumes and scores job fit with an LLM, a rule-based fallback engine, and a responsive React interface with dashboards and charts. The system was tested functionally and on multiple viewports, and deployed to Vercel with MongoDB Atlas.

5.2 Conclusion

The project shows that a hybrid approach — instant deterministic scoring for every job combined with on-demand LLM judgement — gives users both speed and explanation. Job seekers receive specific, actionable feedback on their resume and a ranked list of jobs with the skills they are missing, and recruiters receive a ranked, explained shortlist and can discover candidates who have not applied. Layered validation (HTML5 constraints, JavaScript checks, express-validator and Mongoose schemas), JWT-based role authorisation and the automatic fallback make the application secure and dependable. The project applied the core Web Technology concepts of the course — client-side scripting, component-based UI, HTTP and REST, server-side programming, middleware, and database integration — to a real recruitment problem.

5.3 Future Enhancements

- OCR support (for example Tesseract) so that scanned, image-only resumes can be analysed.
- E-mail and in-app notifications when an application status changes or a new matching job is posted.
- Semantic embeddings stored in a vector index (such as MongoDB Atlas Vector Search) to complement TF-IDF similarity.
- Interview scheduling with calendar integration and AI-generated interview questions based on the missing skills.
- Resume builder that applies the AI suggestions directly and exports an ATS-friendly PDF.

- Automated test suites (Jest / Supertest for the API and Playwright for the UI) running in a CI pipeline on every commit.
- Bias and fairness auditing of AI scores, and an option for recruiters to hide personal details during screening.

REFERENCES

[1] MDN Web Docs, “HTML, CSS and JavaScript reference,” Mozilla. [Online]. Available: https://developer.mozilla.org

[2] W3Schools, “Web development tutorials – HTML, CSS, JavaScript, Node.js, React.” [Online]. Available: https://www.w3schools.com

[3] React Team, “React documentation.” [Online]. Available: https://react.dev

[4] OpenJS Foundation, “Express – Node.js web application framework, API reference.” [Online]. Available: https://expressjs.com

[5] MongoDB Inc., “MongoDB Manual” and “Mongoose ODM documentation.” [Online]. Available: https://www.mongodb.com/docs/manual and https://mongoosejs.com/docs

[6] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.

[7] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[8] OWASP Foundation, “OWASP Top 10 – 2021.” [Online]. Available: https://owasp.org/Top10

[9] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[10] C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[11] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[12] T. Brown et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[13] OpenAI, “gpt-oss-120b & gpt-oss-20b model card,” Aug. 2025.

[14] Groq Inc., “GroqCloud API documentation – chat completions and JSON mode.” [Online]. Available: https://console.groq.com/docs

[15] Ollama, “Ollama documentation and API.” [Online]. Available: https://github.com/ollama/ollama

[16] Mozilla, “PDF.js.” [Online]. Available: https://mozilla.github.io/pdf.js

[17] Chart.js contributors, “Chart.js documentation.” [Online]. Available: https://www.chartjs.org/docs

[18] Vercel Inc., “Vercel documentation – Functions and deployments.” [Online]. Available: https://vercel.com/docs

[19] Project source code: HireSense repository. [Online]. Available: https://github.com/jebarson-caleb/hiresense

SAMPLE OUTPUT SCREENSHOT

A. UI Design

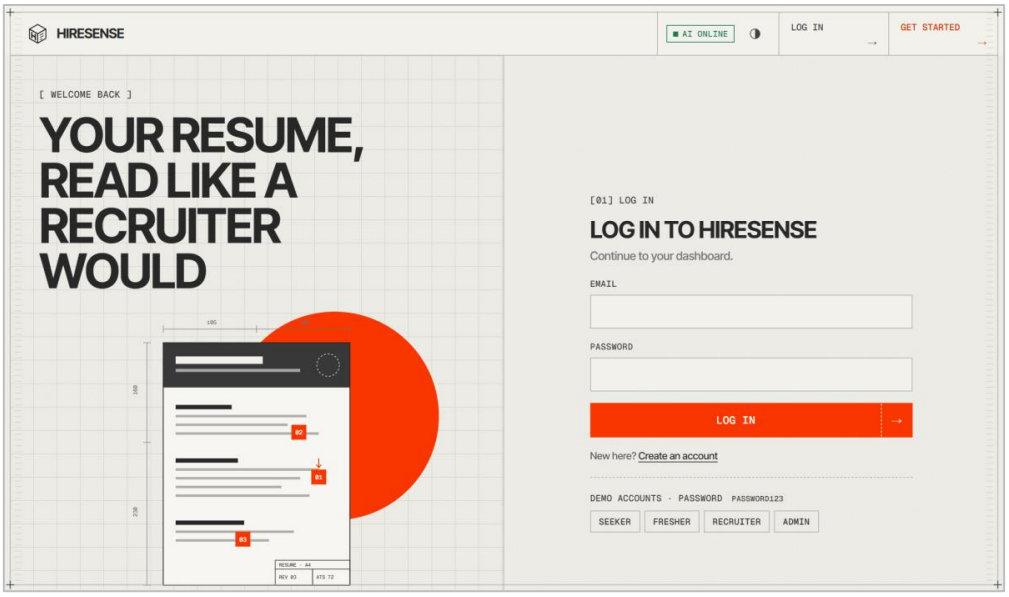


Figure S.1: Sample UI – login page (light theme)

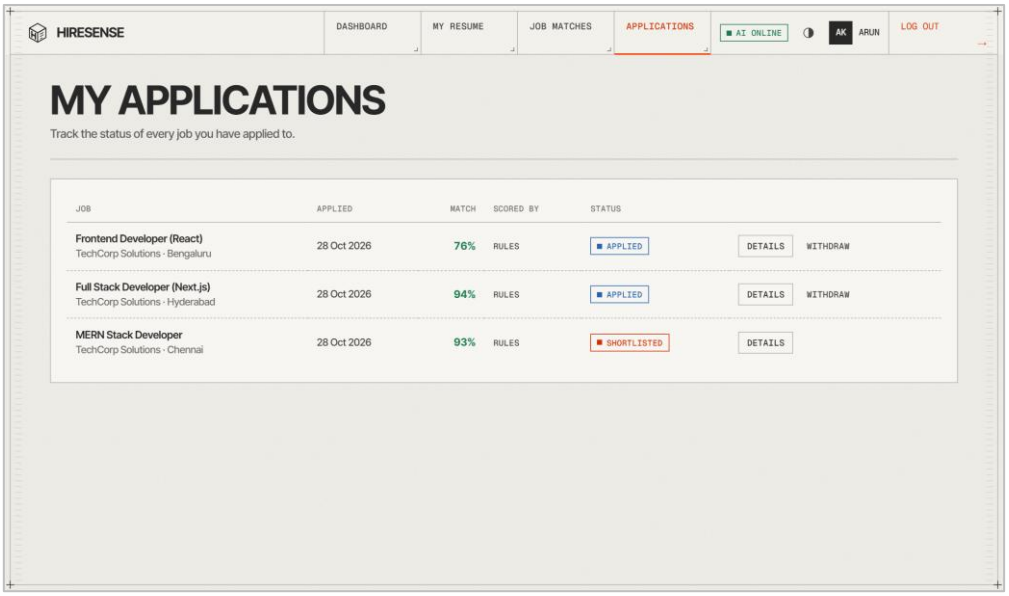


Figure S.2: Sample UI – seeker's applications with match scores and status

B. MongoDB Database Storage

```
mongosh — mongodb://127.0.0.1:27017/resume_portal

resume_portal> show collections
applications   13 documents
jobs           20 documents
resumes        5 documents
users          9 documents
```

Figure S.3: MongoDB database resume_portal – collections and document counts

```
mongosh — mongodb://127.0.0.1:27017/resume_portal

resume_portal> db.resumes.findOne({ fileName: /Arun/ },
                 { fileName: 1, analyzedBy: 1, model: 1, "analysis.atsScore": 1,
                   "analysis.experienceYears": 1, "analysis.skills": { $slice: 8 },
                   "analysis.suggestedRoles": 1 })
{
  _id: ObjectId('6ae216062697c3b59519c626'),
  fileName: 'Arun_Kumar_FullStack.pdf',
  analysis: {
    skills: [
      'JavaScript',
      'TypeScript',
      'HTML',
      'CSS',
      'SQL',
      'React',
      'Redux',
      'Next.js'
    ],
    experienceYears: 3.8,
    suggestedRoles: [
      'Mid-Level Full Stack Engineer',
      'MERN Stack Developer',
      'Software Engineer II'
    ],
    atsScore: 100
  },
  analyzedBy: 'groq',
  model: 'openai/gpt-oss-120b'
}
```

Figure S.4: Stored resume document with AI analysis (resumes collection)

EVALUATION SHEET

Reg.No : URK24CS1038

Name : JEBARSON CALEB D

Course code: 23CS2048

Course Name: Web Technology

S.No	Rubrics	Maximum Marks	Marks Obtained
1	Hackathon	20	
2	Project – Case Study	20	
Total		40	

Rubrics	Excellent (4)	Good (3)	Average (2)	Below Average (1)
Design of Website (4 Marks)				
Input Validation (4 Marks)				
Integration of Frontend and Backend (4 Marks)				
Presentation and Viva (4 Marks)				
GitHub Repository (4 Marks)				

Signature of the Faculty-in-charge

EVALUATION SHEET

Reg.No : URK24CS1025

Name : TALARI S VAMSHI KRISHNA

Course code: 23CS2048

Course Name: Web Technology

S.No	Rubrics	Maximum Marks	Marks Obtained
1	Hackathon	20	
2	Project – Case Study	20	
Total		40	

Rubrics	Excellent (4)	Good (3)	Average (2)	Below Average (1)
Design of Website (4 Marks)				
Input Validation (4 Marks)				
Integration of Frontend and Backend (4 Marks)				
Presentation and Viva (4 Marks)				
GitHub Repository (4 Marks)				

Signature of the Faculty-in-charge

EVALUATION SHEET

Reg.No : URK24CS1118

Name : JASPER FREDDY PAUL R S

Course code: 23CS2048

Course Name: Web Technology

S.No	Rubrics	Maximum Marks	Marks Obtained
1	Hackathon	20	
2	Project – Case Study	20	
Total		40	

Rubrics	Excellent (4)	Good (3)	Average (2)	Below Average (1)
Design of Website (4 Marks)				
Input Validation (4 Marks)				
Integration of Frontend and Backend (4 Marks)				
Presentation and Viva (4 Marks)				
GitHub Repository (4 Marks)				

Signature of the Faculty-in-charge

EVALUATION SHEET

Reg.No : URK24CS1017

Name : PAVIN MUTHU G

Course code: 23CS2048

Course Name: Web Technology

S.No	Rubrics	Maximum Marks	Marks Obtained
1	Hackathon	20	
2	Project – Case Study	20	
Total		40	

Rubrics	Excellent (4)	Good (3)	Average (2)	Below Average (1)
Design of Website (4 Marks)				
Input Validation (4 Marks)				
Integration of Frontend and Backend (4 Marks)				
Presentation and Viva (4 Marks)				
GitHub Repository (4 Marks)				

Signature of the Faculty-in-charge

ANNEXURE: HACKNEX 2026 PARTICIPATION CERTIFICATES

24-Hour National Hackathon organised by the Division of CSE, Karunya Institute of Technology and Sciences

1. JEBARSON CALEB D (URK24CS1038)
2. TALARI S VAMSHI KRISHNA (URK24CS1025)
3. JASPER FREDDY PAUL R S (URK24CS1118)
4. PAVIN MUTHU G (URK24CS1017)

Karunya
DEEMED UNIVERSITY
SOLVING HUMAN PROBLEMS

HACKNEX
NATIONAL HACKATHON 2026

CIRA
COMPUTER INNOVATION AND RESEARCH ASSOCIATION

NEXUS

CERTIFICATE OF APPRECIATION

PROUDLY PRESENTED TO

Jebarson Caleb D

in recognition of the active participation and valuable contribution to **HACKNEX 2026,** a 24-Hour National Hackathon, organized by the Division of **Computer Science and Engineering, Karunya Institute of Technology and Sciences.** Their dedication, innovative spirit, technical aptitude, and commitment to excellence contributed to the success of the event held on **8th-9th October 2026.**

DR. P. EBEN SOPHIA
ORGANISING SECRETARY
ASSISTANT PROFESSOR,
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

DR. G. JASPHER W KATHRINE
CONVENER
PROFESSOR & HOD - CSE,
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

DR. D. NIRMAL
DEAN ACADEMIC AFFAIRS
PROFESSOR & DEAN - AA
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

SPONSORS

KARIMAM GLOBAL VENTURES · Reflexlabs AI · PAYANAM FOUNDATION · AESTRYN · ASIRTECH · NABKISAN

POWERED BY

Google Cloud · Microsoft · EC COUNCIL · SUSE · cisco

Karunya
DEEMED UNIVERSITY
SOLVING HUMAN PROBLEMS

HACKNEX
NATIONAL HACKATHON 2026

CIRA
COMPUTER INNOVATION AND RESEARCH ASSOCIATION

NEXUS

CERTIFICATE OF APPRECIATION

PROUDLY PRESENTED TO

TALARI S VAMSHI KRISHNA

in recognition of the active participation and valuable contribution to HACKNEX 2026, a 24-HourNational Hackathon, organized by the Division of ComputerScienceandEngineering, Karunya Institute of Technology and Sciences. Their dedication, innovative spirit, technical aptitude, and commitment to excellence contributed to the success of the event held on **7th October 2026**.

DR. P. EBEN SOPHIA
ORGANISING SECRETARY
ASSISTANT PROFESSOR,
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

DR. G. JASPHER W KATHRINE
CONVENER
PROFESSOR & HOD - CSE,
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

DR. D. NIRMAL
DEAN ACADEMIC AFFAIRS
PROFESSOR & DEAN - AA
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

SPONSORS

KARIMAM GLOBAL VENTURES · Reflexlabs AI · PAYANAM FOUNDATION · ABSTRYN · ASIRTECH · NABKISAN

POWERED BY

Google Cloud · Microsoft · EC-COUNCIL · SUSE · CISCO

Karunya
DEEMED UNIVERSITY
SOLVING HUMAN PROBLEMS

HACKNEX
NATIONAL HACKATHON 2026

CIRA
COMPUTER INNOVATION AND RESEARCH ASSOCIATION

NEXUS

CERTIFICATE OF APPRECIATION

PROUDLY PRESENTED TO

Jasper Freedy Paul R S

in recognition of the active participation and valuable contribution to HACKNEX 2026, a 24-HourNational Hackathon, organized by the Division of ComputerScienceandEngineering, Karunya Institute of Technology and Sciences. Their dedication, innovative spirit, technical aptitude, and commitment to excellence contributed to the success of the event held on **7th October 2026**.

DR. P. EBEN SOPHIA
ORGANISING SECRETARY
ASSISTANT PROFESSOR,
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

DR. G. JASPHER W KATHRINE
CONVENER
PROFESSOR & HOD - CSE,
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

DR. D. NIRMAL
DEAN ACADEMIC AFFAIRS
PROFESSOR & DEAN - AA
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

SPONSORS

KARIMAM GLOBAL VENTURES · Reflexlabs AI · PAYANAM FOUNDATION · ABSTRYN · ASIRTECH · NABKISAN

POWERED BY

Google Cloud · Microsoft · EC-COUNCIL · SUSE · CISCO

Karunya
DEEMED UNIVERSITY
SOLVING HUMAN PROBLEMS

HACKNEX
NATIONAL HACKATHON 2026

CIRA
COMPUTER INNOVATION AND RESEARCH ASSOCIATION

NEXUS

CERTIFICATE OF APPRECIATION

PROUDLY PRESENTED TO

Pavin Muthu G

in recognition of the active participation and valuable contribution to HACKNEX 2026, a 24-HourNational Hackathon, organized by the Division of ComputerScienceandEngineering, Karunya Institute of Technology and Sciences. Their dedication, innovative spirit, technical aptitude, and commitment to excellence contributed to the success of the event held on **7th October 2026**.

DR. P. EBEN SOPHIA
ORGANISING SECRETARY
ASSISTANT PROFESSOR,
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

DR. G. JASPHER W KATHRINE
CONVENER
PROFESSOR & HOD - CSE,
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

DR. D. NIRMAL
DEAN ACADEMIC AFFAIRS
PROFESSOR & DEAN - AA
KARUNYA INSTITUTE
OF TECHNOLOGY AND SCIENCES

SPONSORS

KARIMAM GLOBAL VENTURES
Reflexlabs AI
PAYANAM FOUNDATION
ABSTRYN
ASIRTECH
NABKISAN

POWERED BY

Google Cloud
Microsoft
EC-COUNCIL
SUSE
CISCO